In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
dataset=pd.read_csv(r"C:\Users\beher\Downloads\Investment.csv")
```

In [3]:
```
dataset.head()
```

Out[3]:

	DigitalMarketing	Promotion	Research	State	Profit
0	165349.20	136897.80	471784.10	Hyderabad	192261.83
1	162597.70	151377.59	443898.53	Bangalore	191792.06
2	153441.51	101145.55	407934.54	Chennai	191050.39
3	144372.41	118671.85	383199.62	Hyderabad	182901.99
4	142107.34	91391.77	366168.42	Chennai	166187.94

In [4]:
```
x=dataset.iloc[:,:-1]#independent variable
y=dataset.iloc[:,4]#depndent variable
```

In [5]:
```
x.head()
```

Out[5]:

	DigitalMarketing	Promotion	Research	State
0	165349.20	136897.80	471784.10	Hyderabad
1	162597.70	151377.59	443898.53	Bangalore
2	153441.51	101145.55	407934.54	Chennai
3	144372.41	118671.85	383199.62	Hyderabad
4	142107.34	91391.77	366168.42	Chennai

In [6]:
```
y.head()
```

Out[6]:
```
0    192261.83
1    191792.06
2    191050.39
3    182901.99
4    166187.94
Name: Profit, dtype: float64
```

In [7]:
```
x=pd.get_dummies(x,dtype=int)
```

In [8]:
```
x.head()
```

Out[8]:

	DigitalMarketing	Promotion	Research	State_Bangalore	State_Chennai	State_Hyderaba
0	165349.20	136897.80	471784.10	0	0	
1	162597.70	151377.59	443898.53	1	0	
2	153441.51	101145.55	407934.54	0	1	
3	144372.41	118671.85	383199.62	0	0	
4	142107.34	91391.77	366168.42	0	1	

In [ ]:

In [9]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)
```

In [10]:
```
from sklearn.linear_model import  LinearRegression
regressor=LinearRegression()
regressor.fit(x_train,y_train)
```

Out[10]:
```
▾ LinearRegression
▶ Parameters
```

In [ ]:

In [11]:
```
y_pred=regressor.predict(x_test)
```

In [12]:
```
intercept=regressor.intercept_
```

In [13]:
```
intercept
```

Out[13]:
```
np.float64(42467.52924855314)
```

In [14]:
```
coeficient=regressor.coef_
```

In [15]:
```
coeficient
```

Out[15]:
```
array([ 7.73467193e-01,  3.28845975e-02,  3.66100259e-02,  8.66383692e+01,
       -8.72645791e+02,  7.86007422e+02])
```

In [ ]:

In [16]:
```
x=np.append(arr=np.full((50,1),42467).astype(int),values=x,axis=1)
```

In [17]:
```
import statsmodels.api as sm
x_opt=x[:,[0,1,2,3,4,5]]
regressor_OLS=sm.OLS(endog=y,exog=x_opt).fit()
regressor_OLS.summary()
```

Out[17]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.951
Model:	OLS	**Adj. R-squared:**	0.945
Method:	Least Squares	**F-statistic:**	169.9
Date:	Thu, 12 Mar 2026	**Prob (F-statistic):**	1.34e-27
Time:	00:04:30	**Log-Likelihood:**	-525.38
No. Observations:	50	**AIC:**	1063.
Df Residuals:	44	**BIC:**	1074.
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1794	0.164	7.204	0.000	0.849	1.509
x1	0.8060	0.046	17.369	0.000	0.712	0.900
x2	-0.0270	0.052	-0.517	0.608	-0.132	0.078
x3	0.0270	0.017	1.574	0.123	-0.008	0.062
x4	41.8870	3256.039	0.013	0.990	-6520.229	6604.003
x5	240.6758	3338.857	0.072	0.943	-6488.349	6969.701

Omnibus:	14.782	**Durbin-Watson:**	1.283
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	21.266
Skew:	-0.948	**Prob(JB):**	2.41e-05
Kurtosis:	5.572	**Cond. No.**	8.45e+05

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 8.45e+05. This might indicate that there are
strong multicollinearity or other numerical problems.

In [ ]:

In [18]:

```
import statsmodels.api as sm
x_opt=x[:,[0,1,2,3,5]]
regressor_OLS=sm.OLS(endog=y,exog=x_opt).fit()
regressor_OLS.summary()
```

Out[18]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.951
Model:	OLS	**Adj. R-squared:**	0.946
Method:	Least Squares	**F-statistic:**	217.2
Date:	Thu, 12 Mar 2026	**Prob (F-statistic):**	8.49e-29
Time:	00:04:41	**Log-Likelihood:**	-525.38
No. Observations:	50	**AIC:**	1061.
Df Residuals:	45	**BIC:**	1070.
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1799	0.157	7.537	0.000	0.865	1.495
x1	0.8060	0.046	17.606	0.000	0.714	0.898
x2	-0.0270	0.052	-0.523	0.604	-0.131	0.077
x3	0.0270	0.017	1.592	0.118	-0.007	0.061
x4	220.1585	2900.536	0.076	0.940	-5621.821	6062.138

Omnibus:	14.758	**Durbin-Watson:**	1.282
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	21.172
Skew:	-0.948	**Prob(JB):**	2.53e-05
Kurtosis:	5.563	**Cond. No.**	6.18e+05

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 6.18e+05. This might indicate that there are
strong multicollinearity or other numerical problems.

In [ ]:

In [19]:

```
import statsmodels.api as sm
x_opt=x[:,[0,1,2,3,]]
regressor_OLS=sm.OLS(endog=y,exog=x_opt).fit()
regressor_OLS.summary()
```

Out[19]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.951
Model:	OLS	**Adj. R-squared:**	0.948
Method:	Least Squares	**F-statistic:**	296.0
Date:	Thu, 12 Mar 2026	**Prob (F-statistic):**	4.53e-30
Time:	00:04:51	**Log-Likelihood:**	-525.39
No. Observations:	50	**AIC:**	1059.
Df Residuals:	46	**BIC:**	1066.
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1803	0.155	7.626	0.000	0.869	1.492
x1	0.8057	0.045	17.846	0.000	0.715	0.897
x2	-0.0268	0.051	-0.526	0.602	-0.130	0.076
x3	0.0272	0.016	1.655	0.105	-0.006	0.060

Omnibus:	14.838	**Durbin-Watson:**	1.282
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	21.442
Skew:	-0.949	**Prob(JB):**	2.21e-05
Kurtosis:	5.586	**Cond. No.**	35.0

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [20]:

```
import statsmodels.api as sm
x_opt=x[:,[0,1]]
regressor_OLS=sm.OLS(endog=y,exog=x_opt).fit()
regressor_OLS.summary()
```

Out[20]:

OLS Regression Results

Dep. Variable:	Profit	**R-squared:**	0.947
Model:	OLS	**Adj. R-squared:**	0.945
Method:	Least Squares	**F-statistic:**	849.8
Date:	Thu, 12 Mar 2026	**Prob (F-statistic):**	3.50e-32
Time:	00:05:17	**Log-Likelihood:**	-527.44
No. Observations:	50	**AIC:**	1059.
Df Residuals:	48	**BIC:**	1063.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.1546	0.060	19.320	0.000	1.034	1.275
x1	0.8543	0.029	29.151	0.000	0.795	0.913

Omnibus:	13.727	**Durbin-Watson:**	1.116
Prob(Omnibus):	0.001	**Jarque-Bera (JB):**	18.536
Skew:	-0.911	**Prob(JB):**	9.44e-05
Kurtosis:	5.361	**Cond. No.**	4.60

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [26]:
```
bias=regressor.score(x_train,y_train)
```

In [27]:
```
variance=regressor.score(x_test,y_test)
```

In [28]:
```
bias
```

Out[28]:
```
0.9501847627493607
```

In [29]:
```
variance
```

Out[29]:
```
0.9347068473282424
```

In [30]:
```
#Overall, the model performs well because bias and variance are balanced.
#The analysis suggests that investing in digital marketing can help achieve maximum
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: